TP N°6: TDD - Test Driven Development



Guía de pruebas a desarrollar y ejecutar para la implementación de la funcionalidad de “Comprar entradas”

GRUPO 5 - 4K1

DOCENTES:
Abdala, Valeria Rosa
Barbera, Juan Salvador
Meles, Silvia Judith

INTEGRANTES:

- Brocanelli, Gaspar
- Comas, Lautaro
- Fassi, Manuel
- Gaitán Acevedo, Baltasar
- Hermoza, Bianca Xiomara
- Imoberdorf, Martín José
- Koroch, Matías Adolfo
- Morellato, Mayra
- Romero Peñaloza, María Lucía
- Salinas, Abril
- Sanchez, Mateo
- Tissera, Luciano
- Vilchez, Felipe

Casos de prueba

CP-01 — Compra exitosa con tarjeta mediante Mercado Pago y mail de confirmación (pasa)
Precondiciones ● El visitante inició sesión con un usuario registrado (ana@mail.com). ● El parque está abierto el jueves 29/10/2026. ● El visitante no compró entradas para esa fecha.
Pasos 1. El visitante ingresa a “Comprar entradas”. 2. El visitante selecciona la fecha de visita 29/10/2026. 3. El visitante ingresa Cantidad de entradas = 2. 4. El visitante ingresa las edades: Visitante 1 = 30, Visitante 2 = 28. 5. El visitante selecciona Tipo de pase = Regular. 6. El visitante selecciona Forma de pago = Tarjeta. 7. El visitante confirma la compra. 8. El visitante paga en Mercado Pago y el pago queda aprobado.
Resultados esperados 9. Al confirmar la compra, el sistema redirige al checkout de Mercado Pago por un monto de $20.000. Todavía no envía el mail. 10. Cuando Mercado Pago informa el pago aprobado, el sistema muestra “Compraste 2 entradas para el 29/10/2026. Total: $20.000”. 11. El sistema envía a ana@mail.com un mail con asunto “Confirmación de compra - EcoHarmony Park”. El mail incluye el código de entrada y el QR. 12. Si se vuelve a confirmar el mismo pago, el mail no se reenvía.

```python
def test_pu1_compra_con_tarjeta_redirige_a_mercado_pago_y_envia_mail(self):
    """PU1: fecha disponible, menos de 10 entradas, edades, pase y tarjeta (pasa)."""
    mercado_pago = ApiMercadoPagoEnMemoria()
    smtp = ServidorSMTPEnMemoria()
    cliente = crear_cliente(crear_servicio(api_mercado_pago=mercado_pago, servidor_smtp=smtp))

    compra = cliente.post(ENDPOINT, json=CUERPO_VALIDO, headers=CABECERAS)

    assert compra.status_code == 201
    assert compra.json()["url_pago"] == URL_CHECKOUT
    assert smtp.enviados == []  # todavía no pagó

    payment_id = mercado_pago.pagar("approved")
    confirmacion = cliente.post(
        ENDPOINT_CONFIRMAR_PAGO, json={"payment_id": payment_id}, headers=CABECERAS
    )

    assert confirmacion.status_code == 200
    datos = confirmacion.json()
    assert datos["mensaje"] == "Compraste 2 entradas para el 29/10/2026. Total: $20.000"
    assert datos["mail_enviado_a"] == USUARIO_REGISTRADO
    assert len(smtp.enviados) == 1
    assert smtp.enviados[0]["To"] == USUARIO_REGISTRADO
```

CP-02 — Compra sin seleccionar forma de pago (falla)

Precondiciones

- El visitante inició sesión con un usuario registrado.
- El parque está abierto el 29/10/2026.

Pasos

1. El visitante selecciona la fecha de visita 29/10/2026.
2. El visitante ingresa Cantidad de entradas = 2.
3. El visitante ingresa las edades: 30 y 28.
4. El visitante selecciona Tipo de pase = Regular.
5. El visitante no selecciona una forma de pago.
6. El visitante confirma la compra.

Resultados esperados

7. El sistema rechaza la compra y muestra el mensaje “Debe seleccionar una forma de pago”.
8. No se registra la compra, no se genera un pago en Mercado Pago y no se envía mail.

```python
def test_pu2_compra_sin_forma_de_pago_falla(self):
    """PU2: comprar entradas sin seleccionar forma de pago (falla)."""
    cliente = crear_cliente()
    cuerpo = {**CUERPO_VALIDO}
    del cuerpo["forma_pago"]

    respuesta = cliente.post(ENDPOINT, json=cuerpo, headers=CABECERAS)

    assert respuesta.status_code == 422
    assert respuesta.json()["detail"] == "Debe seleccionar una forma de pago"
```

CP-03 — Fecha de visita con el parque cerrado (falla)

Precondiciones

- El visitante inició sesión con un usuario registrado.
- El parque cierra los lunes, el 25/12 y el 1/1.
- El 26/10/2026 es lunes.

Pasos

1. El visitante selecciona la fecha de visita 26/10/2026 (lunes).
2. El visitante ingresa Cantidad de entradas = 2.
3. El visitante ingresa las edades: 30 y 28.
4. El visitante selecciona Tipo de pase = Regular.
5. El visitante selecciona Forma de pago = Tarjeta.
6. El visitante confirma la compra.

Resultados esperados

7. El sistema rechaza la compra y muestra el mensaje “El parque está cerrado en la fecha seleccionada”.
8. No se registra la compra, no se redirige a Mercado Pago y no se envía mail.

```python
def test_pu3_compra_con_el_parque_cerrado_falla(self):
    """PU3: fecha de visita en la que el parque está cerrado (falla)."""
    cliente = crear_cliente()
    cuerpo = {**CUERPO_VALIDO, "fecha_visita": "2026-10-26"}  # lunes

    respuesta = cliente.post(ENDPOINT, json=cuerpo, headers=CABECERAS)

    assert respuesta.status_code == 422
    assert respuesta.json()["detail"] == "El parque está cerrado en la fecha seleccionada"
```

CP-04 — Cantidad de entradas mayor a 10 (falla)

Precondiciones

- El visitante inició sesión con un usuario registrado.
- El parque está abierto el 29/10/2026.

Pasos

1. El visitante selecciona la fecha de visita 29/10/2026.
2. El visitante ingresa Cantidad de entradas = 11.
3. El visitante ingresa la edad de los 11 visitantes (30 años cada uno).
4. El visitante selecciona Tipo de pase = Regular.
5. El visitante selecciona Forma de pago = Tarjeta.
6. El visitante confirma la compra.

Resultados esperados

7. El sistema rechaza la compra y muestra el mensaje “La cantidad de entradas no puede superar 10”.
8. No se registra la compra, no se genera un pago y no se envía mail.

```python
def test_pu4_compra_de_mas_de_10_entradas_falla(self):
    """PU4: cantidad de entradas mayor a 10 (falla)."""
    cliente = crear_cliente()
    cuerpo = {**CUERPO_VALIDO, "cantidad": 11, "edades": [30] * 11}

    respuesta = cliente.post(ENDPOINT, json=cuerpo, headers=CABECERAS)

    assert respuesta.status_code == 422
    assert respuesta.json()["detail"] == "La cantidad de entradas no puede superar 10"
```

CP-05 — Compra exitosa en efectivo con distintas edades (pasa)

Precondiciones

- El visitante inició sesión con un usuario registrado.
- El parque está abierto el 29/10/2026.

Pasos

1. El visitante selecciona la fecha de visita 29/10/2026.
2. El visitante ingresa Cantidad de entradas = 4.
3. El visitante ingresa las edades: 35, 33, 10 y 2.
4. El visitante selecciona Tipo de pase = Regular.
5. El visitante selecciona Forma de pago = Efectivo.
6. El visitante confirma la compra.

Resultados esperados

7. El sistema registra la compra correctamente.
8. El sistema calcula el total: $10.000 + $10.000 (adultos) + $5.000 (menor, 50%) + $0 (menor de 3 años) = $25.000.

```python
def test_informa_el_monto_total_segun_tipo_de_pase_y_edades(self):
    servicio = crear_servicio()
    solicitud = SolicitudCompra(
        usuario_email="ana@mail.com",
        fecha_visita=FECHA_VALIDA,
        cantidad=4,
        edades=[35, 33, 10, 2],
        tipo_pase=TipoPase.REGULAR,
        forma_pago=FormaPago.EFECTIVO,
    )

    resultado = servicio.comprar(solicitud)

    assert resultado.exitosa is True
    assert resultado.monto_total == 25000
```

CP-06 — Compra con un usuario no registrado (falla)

Precondiciones

- El email intruso@mail.com no está registrado en el sistema.
- El parque está abierto el 29/10/2026.

Pasos

1. El visitante intenta comprar con el email intruso@mail.com.
2. El visitante selecciona la fecha de visita 29/10/2026.
3. El visitante ingresa Cantidad de entradas = 2 y las edades 30 y 28.
4. El visitante selecciona Tipo de pase = Regular y Forma de pago = Efectivo.
5. El visitante confirma la compra.

Resultados esperados

6. El sistema rechaza la operación (acceso denegado, HTTP 403) y muestra el mensaje “Debe estar registrado para comprar entradas”.
7. No se registra la compra ni se envía mail.

```python
def test_no_deberia_comprar_un_usuario_no_registrado(self):
    servicio = crear_servicio()
    solicitud = SolicitudCompra(
        usuario_email="intruso@mail.com",
        fecha_visita=FECHA_VALIDA,
        cantidad=2,
        edades=[30, 28],
        tipo_pase=TipoPase.REGULAR,
        forma_pago=FormaPago.EFECTIVO,
    )

    resultado = servicio.comprar(solicitud)

    assert resultado.exitosa is False
    assert resultado.error == "Debe estar registrado para comprar entradas"
```

CP-07 — Fecha de visita anterior al día actual (falla)

Precondiciones

- El visitante inició sesión con un usuario registrado.
- Fecha actual: 22/10/2026.

Pasos

1. El visitante selecciona la fecha de visita 21/10/2026.
2. El visitante ingresa Cantidad de entradas = 2 y las edades 30 y 28.
3. El visitante selecciona Tipo de pase = Regular y Forma de pago = Efectivo.
4. El visitante confirma la compra**.**

Resultados esperados

5. El sistema rechaza la compra y muestra el mensaje “La fecha de visita debe ser hoy o una fecha futura”.
6. No se registra la compra ni se envía mail.

```python
def test_rechaza_otra_fecha_pasada(self):
    servicio = crear_servicio()
    solicitud = SolicitudCompra(
        usuario_email="ana@mail.com",
        fecha_visita=HOY - timedelta(days=1),
        cantidad=2,
        edades=[30, 28],
        tipo_pase=TipoPase.REGULAR,
        forma_pago=FormaPago.EFECTIVO,
    )

    resultado = servicio.comprar(solicitud)

    assert resultado.exitosa is False
    assert resultado.error == "La fecha de visita debe ser hoy o una fecha futura"
```

CP-08 — Compra para el día actual (pasa)

Precondiciones

- El visitante inició sesión con un usuario registrado.
- Fecha y hora actual: jueves 22/10/2026, 10:00 (antes de las 17:30, hora límite para comprar para el mismo día).
- El parque está abierto ese día.

Pasos

1. El visitante selecciona la fecha de visita 22/10/2026 (hoy).
2. El visitante ingresa Cantidad de entradas = 2 y las edades 30 y 28.
3. El visitante selecciona Tipo de pase = Regular y Forma de pago = Efectivo.
4. El visitante confirma la compra.

Resultados esperados

5. El sistema registra la compra correctamente.

```python
def test_permite_comprar_para_el_dia_de_hoy(self):
    servicio = crear_servicio()
    solicitud = SolicitudCompra(
        usuario_email="ana@mail.com",
        fecha_visita=HOY,
        cantidad=2,
        edades=[30, 28],
        tipo_pase=TipoPase.REGULAR,
        forma_pago=FormaPago.EFECTIVO,
    )

    resultado = servicio.comprar(solicitud)

    assert resultado.exitosa is True
```

CP-09 — Compra sin seleccionar tipo de pase (falla)
Precondiciones ● El visitante inició sesión con un usuario registrado. ● El parque está abierto el 29/10/2026.
Pasos 1. El visitante selecciona la fecha de visita 29/10/2026. 2. El visitante ingresa Cantidad de entradas = 2 y las edades 30 y 28. 3. El visitante no selecciona un tipo de pase. 4. El visitante selecciona Forma de pago = Efectivo. 5. El visitante confirma la compra.
Resultados esperados 6. El sistema rechaza la compra y muestra el mensaje “Debe seleccionar un tipo de pase”. 7. No se registra la compra ni se envía mail.

```python
def test_no_deberia_comprar_sin_tipo_de_pase(self):
    servicio = crear_servicio()
    solicitud = SolicitudCompra(
        usuario_email="ana@mail.com",
        fecha_visita=FECHA_VALIDA,
        cantidad=2,
        edades=[30, 28],
        tipo_pase=None,
        forma_pago=FormaPago.EFECTIVO,
    )

    resultado = servicio.comprar(solicitud)

    assert resultado.exitosa is False
    assert resultado.error == "Debe seleccionar un tipo de pase"
```

CP-10 — Falta la edad de un visitante (falla)

Precondiciones

- El visitante inició sesión con un usuario registrado.
- El parque está abierto el 29/10/2026.

Pasos

1. El visitante selecciona la fecha de visita 29/10/2026.
2. El visitante ingresa Cantidad de entradas = 3.
3. El visitante ingresa solo dos edades (30 y 28) y deja vacía la del Visitante 3.
4. El visitante selecciona Tipo de pase = Regular y Forma de pago = Efectivo.
5. El visitante confirma la compra.

Resultados esperados

6. El sistema rechaza la compra y muestra el mensaje “Debe indicar la edad de cada visitante”.
7. No se registra la compra ni se envía mail.

```python
def test_rechaza_si_falta_la_edad_de_algun_visitante(self):
    servicio = crear_servicio()
    solicitud = SolicitudCompra(
        usuario_email="ana@mail.com",
        fecha_visita=FECHA_VALIDA,
        cantidad=3,
        edades=[30, 28],
        tipo_pase=TipoPase.REGULAR,
        forma_pago=FormaPago.EFECTIVO,
    )

    resultado = servicio.comprar(solicitud)

    assert resultado.exitosa is False
    assert resultado.error == "Debe indicar la edad de cada visitante"
```

CP-11 — Compra de exactamente 10 entradas (pasa)

Precondiciones

- El visitante inició sesión con un usuario registrado.
- El parque está abierto el 29/10/2026.
- El visitante no compró entradas para esa fecha.

Pasos

1. El visitante selecciona la fecha de visita 29/10/2026.
2. El visitante ingresa Cantidad de entradas = 10.
3. El visitante ingresa la edad de los 10 visitantes (30 años cada uno).
4. El visitante selecciona Tipo de pase = Regular y Forma de pago = Efectivo.
5. El visitante confirma la compra.

Resultados esperados

6. El sistema acepta la compra, porque 10 es el límite permitido.

```python
def test_permite_exactamente_10_entradas(self):
    servicio = crear_servicio()
    solicitud = SolicitudCompra(
        usuario_email="ana@mail.com",
        fecha_visita=FECHA_VALIDA,
        cantidad=10,
        edades=[30] * 10,
        tipo_pase=TipoPase.REGULAR,
        forma_pago=FormaPago.EFECTIVO,
    )

    resultado = servicio.comprar(solicitud)

    assert resultado.exitosa is True
```

CP-12 — Superar 10 entradas sumando compras de la misma fecha (falla)

Precondiciones

- El visitante inició sesión con un usuario registrado.
- El visitante ya compró 8 entradas para el 29/10/2026.

Pasos

1. El visitante selecciona la fecha de visita 29/10/2026.
2. El visitante ingresa Cantidad de entradas = 3 y las edades de los 3 visitantes.
3. El visitante selecciona Tipo de pase = Regular y Forma de pago = Efectivo.
4. El visitante confirma la compra.

Resultados esperados

5. El sistema rechaza la compra y muestra el mensaje “Solo puede comprar hasta 10 entradas por fecha. Ya compró 8 para ese día”.
6. No se registra la nueva compra.

```python
def test_no_deberia_superar_10_entradas_sumando_compras_de_la_misma_fecha(self):
    servicio = crear_servicio()
    primera_compra = SolicitudCompra(
        usuario_email="ana@mail.com",
        fecha_visita=FECHA_VALIDA,
        cantidad=8,
        edades=[30] * 8,
        tipo_pase=TipoPase.REGULAR,
        forma_pago=FormaPago.EFECTIVO,
    )
    segunda_compra = SolicitudCompra(
        usuario_email="ana@mail.com",
        fecha_visita=FECHA_VALIDA,
        cantidad=3,
        edades=[30] * 3,
        tipo_pase=TipoPase.REGULAR,
        forma_pago=FormaPago.EFECTIVO,
    )
    servicio.comprar(primera_compra)

    resultado = servicio.comprar(segunda_compra)

    assert resultado.exitosa is False
    assert resultado.error == (
        "Solo puede comprar hasta 10 entradas por fecha. Ya compró 8 para ese día"
    )
```

CP-13 — Pago con tarjeta no aprobado por Mercado Pago (falla)

Precondiciones

- El visitante inició sesión con un usuario registrado.
- El visitante completó una compra válida con Forma de pago = Tarjeta y fue redirigido a Mercado Pago.

Pasos

1. El visitante intenta pagar en Mercado Pago y el pago queda rechazado, pendiente, en proceso o cancelado.
2. Mercado Pago devuelve al visitante a la página de “Compra finalizada”.

Resultados esperados

3. El sistema consulta el estado real del pago a Mercado Pago y muestra el mensaje “El pago no fue aprobado por Mercado Pago”.
4. No se envía el mail de confirmación.

```python
@pytest.mark.parametrize("estado", ["rejected", "pending", "in_process", "cancelled"])
def test_si_el_pago_no_esta_aprobado_no_envia_mail(self, estado):
    self.servicio.comprar(compra_con_tarjeta())
    payment_id = self.mercado_pago.pagar(estado)

    resultado = self.servicio.confirmar_pago(payment_id, USUARIO_REGISTRADO)

    assert resultado.exitosa is False
    assert resultado.error == "El pago no fue aprobado por Mercado Pago"
    assert self.smtp.enviados == []
```

Conversación con el PO:

Hola Profe, somos del grupo 5 de la materia ISW del curso 4K1, y tenemos algunas preguntas para hacerle sobre la implementación del producto y detalles de los requerimientos.

1- Dado que el enunciado plantea el desarrollo de una aplicación móvil, queríamos consultarles si sería posible desarrollar una Aplicación Web Progresiva (PWA) para demostrar su funcionamiento y cumplir con los requisitos solicitados.

Si puede ser una PWA MOBILE FIRST. Les recomiendo IONIC, funciona rápido y cero problema con permisos de los dispositivos y tiene una gran librería de componentes reutilizables :).
Saludos!

2- Cuando el visitante selecciona pago en efectivo, la compra queda confirmada o pendiente de pago?

Queda pendiente de pago, se paga luego en boletería, pero queda la "reserva" de la entrada hecha.

3- El front sólo debe limitarse a la US o le agregamos al menos una página de Inicio.
Sean creativos! Por más que solo evaluemos una parte, no les va a llevar nada hacer un front base desde el cual acceder a la user.

4- A qué se refieren con "desarrollo" en lo relacionado al envío de mail y a MP?
Que se espera que lo programen para obtener nota alta. Pero si se les complica, pueden simularlo como ventanas a parte siguiendo las condiciones de la user. Les desafío a que usen todas las herramientas que tienen para aprender algo nuevo!

5- Quería consultar acerca del logo, que en el archivo FAQ dice que debe ser validado. Veo que no se mandó la imagen, solo sale image.png.
El logo fue validado posteriormente.